associate it with a given route by calling `Map()` on the `IEndpointRouteBuilder` instance and passing in a route template.

Listing 31.10 Mapping the ping-pong endpoint in `UseEndpoints`

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
var endpoint = ((IEndpointRouteBuilder)app)                    ❶
    .CreateApplicationBuilder()                                ❷
    .UseMiddleware<PingPongMiddleware>()                       ❸
    .Build();                                                  ❸

app.Map("/ping", endpoint);                                    ❹
app.MapRazorPages();
app.MapHealthChecks("/healthz");
app.Run();
```

❶ Casts the WebApplication to IEndpointRouteBuilder so you can call CreateApplicationBuilider

❷ Creates a miniature, standalone IApplicationBuilder to build your endpoint

❸ Adds the middleware and builds the final endpoint. This is executed when the endpoint is executed.

❹ Maps the new endpoint with the route template "/ping"

TIP Note that the `Map()` function on `IEndpointRouteBuilder` creates a new endpoint (consisting of your mini-pipeline) with an associated route. Although it has the same name, this is conceptually different from the `Map` function on `IApplicationBuilder` from section 31.1.2, which is used to branch the middleware pipeline. It is analogous to the `MapGet` (and kin) methods you use to create minimal API endpoints.

As is common with ASP.NET Core, you can extract this somewhat-verbose functionality into an extension method to make your endpoint easier to read and discover. The following listing extracts the code to create an endpoint